AI-API Intelligence

Project Journey Notes — Step-by-Step Walkthrough (Steps 1–31)

This note documents the complete, verified project journey based on the notebook **01_inspect_raw_data.ipynb**. It walks through data inspection, cleaning, category consolidation, and the full modeling experimentation path — from TF-IDF and Logistic Regression through Linear SVM, sentence embeddings, and DistilBERT — ending with the best-performing model. Corrections to the original notes are marked clearly.

Step 1: Load the Raw API Data and Check Its Shape

- Loaded the raw CSV/API data into a Pandas DataFrame.
- The dataset had **1,737 rows** and **7 columns**: name, description, category, auth, https, cors, link.

```
df.shape
# Output: (1737, 7)
```

Step 2: Check for Missing Values

- Used df.isnull().sum() to check for missing data.
- Result: name, description, category, auth, https, cors, link → all 0 missing values.
- So there were no missing values in the original data.

```
df.isnull().sum()
```

Step 3: Understand the Data Using df.describe()

- Since most columns are categorical/text, describe() showed total count, unique values, most common value, and frequency of the most common value.
- Examples: name → 1,716 unique | description → 1,683 unique | category → 51 unique | auth → 7 unique | link → 1,734 unique.

```
df.describe()
```

Step 4: Check Duplicate Names

- Checked duplicate values in the name column and found **42 rows** with repeated names.
- These were not necessarily exact duplicates — e.g. 'Bhagavad Gita' appeared twice with different descriptions and links.
- Conclusion: same name does not mean same API record, so these rows were not simply dropped.

```
duplicates = df[df.duplicated(subset='name', keep=False)]
```

Step 5: Convert https from Yes/No to True/False

- Converted the https column values (Yes: 1645, No: 92) into boolean True/False.
- After conversion, the column datatype became bool.

```
df["https"] = df["https"].map({
    "Yes": True,
    "No": False
})
```

Note: It was the **https** column — not the auth column — that originally contained Yes/No values.

Step 6: Check Value Counts for auth and category

- auth value counts: No → 819, apiKey → 761, OAuth → 150, X-Mashape-Key → 6, User-Agent → 1.
- So auth was not Yes/No — it contained different authentication methods.
- Also checked df['category'].value_counts(); there were 51 categories initially.

```
df['category'].value_counts()
```

Step 7: Check the Relationship Between Category and Authentication

- Used a crosstab to see which authentication methods are used within each API category.
- Example: the Development category used No, OAuth, X-Mashape-Key, and apiKey.

```
pd.crosstab(df['category'], df['auth'])
```

Step 8: Sort the Category Counts

- Sorted category counts ascending to identify the smallest categories.
- Examples: Events (3), Patent (4), Programming (5), Continuous Integration (6), Authentication & Authorization (7), Phone (7), Open Source Projects (9), Vehicle (10), Data Validation (10), Tracking (11).
- This showed several categories had very few examples.

```
df['category'].value_counts().sort_values()
```

Step 9: Combine Very Small Categories into 'Other'

- Decision rule: if a category has fewer than 15 examples, group it into 'Other'.
- This reduced the number of classes, making the classification problem more manageable.
- Categories with 15+ examples stayed as-is; those with fewer became 'Other'.
- Blockchain, with exactly 15 examples, stayed as its own category.

```
category_counts = df['category'].value_counts()

small_categories = category_counts[category_counts < 15].index

df['category'] = df['category'].where(
    ~df['category'].isin(small_categories),
    'Other'
)
```

Step 10: Check the Number of Unique Categories

- Checked the reduced category count.
- Result: **41** unique categories — reduced from the original 51.

```
df['category'].nunique()
# Output: 41
```

Step 11: Split Records into Train, Validation, and Test

- Used a 70% / 15% / 15% stratified split.
- Actual output — Train: 1215, Validation: 261, Test: 261 (Total: 1737).
- Achieved via test_size=0.3 with stratification, then splitting the remaining 30% into two equal halves.

```
train_test_split(..., test_size=0.3, stratify=df['category'])
```

Note: The actual train size was **1,215**, not 1,216 as originally noted.

Step 12: Convert Text into Numbers Using TF-IDF

- Used TfidfVectorizer with max_features=2000 and English stop words removed.
- TF-IDF gives more importance to words useful for identifying a category, and less importance to very common/unimportant words.
- Resulting shapes — Train: (1215, 2000), Validation: (261, 2000), Test: (261, 2000).

```
TfidfVectorizer(
    max_features=2000,
    stop_words='english'
)
```

Step 13: Fit TF-IDF Only on Training Data

- Used fit_transform() only on the training set, and transform() only on validation/test.
- This prevents data leakage — the model should not learn vocabulary/statistics from validation or test data before evaluation.

```
X_train = vectorizer.fit_transform(train_df['description'])

X_val = vectorizer.transform(val_df['description'])
X_test = vectorizer.transform(test_df['description'])
```

Step 14: Train the First Logistic Regression Model

- First classifier: TF-IDF features + Logistic Regression.

```
model = LogisticRegression(
    max_iter=1000,
    random_state=42
)

model.fit(X_train, train_df['category'])
```

Step 15: Check Precision, Recall, F1-score and Support

- Generated a classification_report on the validation set.
- First validation accuracy: **42.53%**.
- Several small classes showed zero precision/recall because the model failed to predict them correctly.

```
classification_report(
    val_df['category'],
    val_predictions
)
```

Step 16: Inspect the Training Data to Understand the Problem

- Manually inspected examples from categories such as Health and Anti-Malware.
- Health contained many COVID/medical-related descriptions; Anti-Malware contained malware/security-related ones.
- Insight: categories can have overlapping language, which makes classification difficult.

Step 17: Create a Logistic Regression Model with Class Balancing

- Applied class_weight='balanced' to give more importance to smaller classes.
- Validation accuracy improved to **48.28%** (up from 42.53%).

```
LogisticRegression(
    max_iter=1000,
    random_state=42,
    class_weight='balanced'
)
```

Step 18: Add Both Name and Description as Text Features

- Combined name and description into a single text_features column instead of using description alone.
- Trained TF-IDF on the combined text features.
- Result: Train accuracy 88.15%, Test accuracy 56.32%.
- A large gap between training and evaluation performance indicated overfitting.

```
train_df['text_features'] = (
    train_df['name'] + ' ' + train_df['description']
)
```

Experimentation Journey

Step 19: Lower Logistic Regression C to 0.5

- Tried stronger regularization with C=0.5.
- Result: Train 84.28%, Validation 48.28% — validation did not improve.

```
C=0.5
```

Step 20: Lower C to 0.1

- Tried even stronger regularization with C=0.1.
- Result: Train 79.09%, Validation 45.21% — this was worse.
- Conclusion: more regularization did not solve the overfitting problem.

```
C=0.1
```

Step 21: Reduce TF-IDF Features

- Experimented with fewer TF-IDF features to simplify the model and reduce overfitting.
- This did not provide a useful validation improvement, so the approach was not continued.

Step 22: Try Bigrams with min_df=2

- Changed TF-IDF to use unigrams + bigrams with min_df=2.
- Result: Train 83.05%, Validation 51.72% — only a small improvement.

```
ngram_range=(1,2)
min_df=2
```

Step 23: Try Unigrams with Sublinear TF

- Enabled sublinear_tf=True.
- Result: Train 88.40%, Validation 51.34% — almost the same as the TF-IDF baseline.
- Conclusion: sublinear TF did not solve the main problem.

```
sublinear_tf=True
```

Step 24: Switch from Logistic Regression to Linear SVM

- Tried LinearSVC with class_weight='balanced'.
- Validation improved to **56.32%**, but training accuracy was 99.34%.
- The model was still heavily overfitting.

```
LinearSVC(
    class_weight='balanced'
)
```

Step 25: Switch from TF-IDF to Sentence Embeddings

- Used the all-MiniLM-L6-v2 sentence embedding model, producing 384-dimensional vectors per text.
- Result: Train 80.33%, Validation 60.54% — a major improvement.
- This also reduced the train-validation gap compared to the earlier TF-IDF model.

Step 26: SVM + Sentence Embeddings

- Combined sentence embeddings with Linear SVM.
- Result: Train 95.88%, Validation 62.84%.
- SVM performed better than Logistic Regression on the same embeddings.

Step 27: Lower SVM C to 0.5

- Tried LinearSVC(class_weight='balanced', C=0.5).
- Result: Train 92.76%, Validation 63.98% — improved validation and reduced the train-validation gap.

```
LinearSVC(
    class_weight='balanced',
```

```
    C=0.5
)
```

Step 28: Switch to MPNet Embeddings

- Used SentenceTransformer('all-mpnet-base-v2'), producing 768-dimensional vectors.
- Shapes — Train: (1215, 768), Validation: (261, 768), Test: (261, 768).
- With SVM + MPNet: Train 93.58%, Validation 68.58% — the best validation result so far.

```
SentenceTransformer('all-mpnet-base-v2')
```

Step 29: Sweep Different SVM C Values

- Tested C = 0.1 → 65.52%, C = 0.3 → 68.58%, C = 0.7 → 68.20%, C = 1.0 → 68.20%, C = 2.0 → 67.82%.
- C = 0.3 was the best among those tested, with no meaningful improvement beyond the 0.3–0.5 region.

Step 30: Add Metadata Features

- Added https, cors, and auth alongside the MPNet embeddings, for a combined 775 features.
- Result: Train 93.99%, Validation 68.20% — a slight drop from 68.58%.
- Conclusion: adding metadata did not help the model.

Step 31: Try Fine-Tuning DistilBERT

- Trained an end-to-end DistilBERT model for 3 epochs.
- Validation accuracy per epoch — Epoch 1: 43.30%, Epoch 2: 48.28%, Epoch 3: 52.49%.
- Final validation accuracy: 52.49%, much worse than SVM + MPNet (68.58%).
- Conclusion: DistilBERT fine-tuning was not the best approach for this dataset.

Final Conclusion

Raw API Data → Data Inspection → Handle duplicates / understand data → Convert Yes/No fields → Analyze category distribution → Small categories → Other → 41 categories → Train / Validation / Test split → TF-IDF → Logistic Regression → Low validation accuracy → Try class balancing → Try different C values → Try bigrams → Try sublinear TF → Linear SVM → Sentence Embeddings → SVM + Embeddings → C = 0.5 → MPNet Embeddings → SVM + MPNet → **68.58% validation** → Try metadata (slightly worse) → Try DistilBERT (52.49%) → **Final choice: SVM + MPNet**

Important Corrections to Original Notes

Correction 1 — TF-IDF + Logistic Regression results: The original note stated "Logistic Regression with TF-IDF got 88.15% train and 51% validation." The notebook actually shows **88.15% train and 56.32% test** for the name + description TF-IDF Logistic Regression experiment. The earlier basic TF-IDF validation accuracy was 42.53%, and the class-balanced version reached 48.28%.

Correction 2 — Yes/No column: auth was not the Yes/No column. It was **https** that contained Yes/No values. The **auth** column instead contained authentication method values: No, apiKey, OAuth, X-Mashape-Key, and User-Agent.

Correction 3 — Train set size: The actual train size from the 70/15/15 stratified split was **1,215** records, not 1,216 as originally noted.

Overall, the original project journey understanding was very close to correct. With these three corrections applied, the explanation is technically accurate and ready for an interview setting.